

RegEx Basics

 [_ \(https://github.com/learn-co-curriculum/python-p3-regex-basics\)](https://github.com/learn-co-curriculum/python-p3-regex-basics)  [_ \(https://github.com/learn-co-curriculum/python-p3-regex-basics/issues/new\)](https://github.com/learn-co-curriculum/python-p3-regex-basics/issues/new)

Learning Goals

- Build patterns using regular expressions.

Key Vocab

- **Regular Expression:** a sequence of characters used to search for a pattern inside of a string.
- **Pattern:** a description of sequences of characters that share certain traits with one another. Sequences do not need to be the same length or share any common characters to pattern match. Also called a **filter**.

Introduction

In this lesson, we're going to learn the syntax and basic vocabulary of regular expressions. We'll start simple and build from there. A great place to head for RegEx testing and practice is [regex101](https://regex101.com/)  [_ \(https://regex101.com/\)](https://regex101.com/) - it allows you to build and test regular expressions against text that you define. In a separate window, open up *regex101*. In the text box labeled "insert your test string here", paste in the following monologue from Shakespeare's *The Merchant of Venice*:

If to do were as easy as to know what were good to do, chapels had been House of worships and poor men's cottages princes' palaces. It is a good divine that follows his own instructions: I can easier teach twenty what were good to be done, than be one of the twenty to follow mine own teaching. The brain may devise laws for the blood, but a hot temper leaps o'er a cold decree: such a hare is madness the youth, to skip o'er the meshes of good counsel the cripple. But this reasoning is not in the fashion to choose me a Spouse or partner. O me, the word

'choose!' I may neither choose whom I would nor refuse whom I dislike; so is the will of a living daughter curbed by the will of a dead father. Is it not hard, Nerissa, that I cannot choose one nor refuse none?

Your window should look like this:



Writing Regular Expressions

In Python, regular expressions require you to use the `re` module from the standard library. Remember that when we say that a module comes from the *standard library*, it means that it was downloaded onto our computer when we installed Python. We still need to `import` it, but we do not need to include it in our Pipfile.

All regular expressions in Python begin with an "r" before the pattern to be matched:

```
pattern = r'abc'
```

This "r" stands for *raw*, which means that escape characters such as backslashes (`\`) are read and not ignored. This expands the number of characters that can go into a pattern and allows you to search for patterns with greater flexibility.

Simple Text Matching

Let's start with the simplest text matching. Select `</> Python` in the "Flavor" column on the left and enter the following RegEx (regex101 will provide the "r" and quotes for you):

```
r'twenty'
```

Notice that the pattern matches the two instances of "twenty" in the passage. Writing a series of letters or numbers in your regular expression will result in a search for exact matches of this pattern anywhere in the string.

Metacharacters

The real beauty of regular expressions is revealed in their use of metacharacters. Metacharacters allow you to use a pre-defined shorthand to match specific characters. For example, `\d` will match any digit in your text, and `\w` will match any word character (letters, numbers, and underscores). The 'RegEx Quick Reference' at the bottom-right corner of regex101 shows metacharacters and patterns that you can use. Play around with these a little. Use `\W` (notice uppercasing) to match the non-word characters in your text.

► *Which character in the constructor of a RegEx pattern allows the interpreter to read backslashes?*

Only specific characters

If I want to match all instances of vowels in a string, the RegEx `r'aeiou'` won't work (feel free to try it), as it will only match the entire string "aeiou" - which clearly isn't in our text. Instead let's use square brackets: `r'[aeiou]'` - this is looking for only **one single** character in our text which matches any of the characters inside the square brackets. If you add this RegEx to our regex101, you'll see every vowel highlighted in your match result.

Ranges

Based on what we've just learned, we can write a regular expression looking for single characters in the first 10 letters of the alphabet like so: `r'[abcdefghij]'` We can actually shorten this in Python using a RegEx range: `r'[a-j]'` .

`r'[0123456789]'` becomes `r'[0-9]'` .

A useful range to remember is `r'[A-z]'` . This represents all letters, both capital and lowercase.

Example: Double Vowels

There are many other metacharacters and ways of building patterns in RegEx, many of which you can refer in the regex101 quick reference guide. However, the best way to actually learn to use regular expressions is to practice building your own patterns. Let's look for instances in our text of two consecutive vowels (for example, 'ae', 'ie', 'oo', etc). The longest way to do this is to hand code the different combinations of two vowels: `r'aalaelailaolaulealeeleileoleulialie'` . It's pretty tedious to hand code each of these combinations (*I certainly didn't finish*). An improvement is to use two sets of square brackets with vowels, each one representing a single character: `r'[aeiou][aeiou]'` . Our most efficient, however, is to use repetitions: `r'[aeiou]{2}'` The curly braces surrounding mean that the pattern or character directly preceding it must repeat that number of times. As such, we're looking for a repeat of a vowel two times. As you can see, there are many ways to write a regular expression that does the same thing.

► *How would you write a RegEx to search for two digit numbers with neither number greater than 5?*

Conclusion

In this lesson, we have explored the basics of constructing RegEx patterns in Python. In subsequent lessons, we will put these rules into action as we explore the methods of the `re` module in Python's standard library.

For more practice, return to the earlier monologue from *The Merchant of Venice*: with a bit of dedication, you should be able to match the whole excerpt!

NOTE: You could certainly match the whole monologue with the simple RegEx `r'.*'`. But where's the fun in that?

Resources

- [re - Regular expression operations - Python](https://docs.python.org/3/library/re.html)  [\(https://docs.python.org/3/library/re.html\)](https://docs.python.org/3/library/re.html)
- [regex101](https://regex101.com/)  [\(https://regex101.com/\)](https://regex101.com/)